

Cabinet-BNN 技术路线

以 HSH-64 权重码为参数载体的二值激活语言模型

v2 • 取代 v1 (v1 的「冻结全精度主干」假设已作废)

2026-02-12

文档性质：架构设计规范 + 工程实施路线。

前置依赖：本文档的 §2 基于 F:\HSH_64 快照的源码阅读；作者已确认该快照不是最新代码，拿到权威版本后须复核（见 §2 复核清单）。

主要未决项：D1-D7（见 §14）。其中 **D7**（激活二值化下的精度补偿）是唯一可能影响架构可行性的未决项。

目录

1	定位与总览	2
1.1	一句话定位	2
1.2	与 v1 的差异	2
1.3	HSH-64 原始设计（作者论文）	2
2	参考实现：快照级约定	2
2.1	Deep Hash 二进制格式（导出侧必须逐字节对齐）	3
3	权重码架构（本设计的核心）	3
3.1	128 位权重码	3
3.2	三层语义对齐（「无缝衔接」的精确定义）	3
3.3	容量分析（修正 v1 的错误结论）	4
3.4	权重生成	4
3.4.1	符号合成： $s = \text{mask} \odot p$	5
3.4.2	几何隔离性质（本设计的数学基石）	5
3.5	param 的结构化初始化	5
3.6	表达力边界（须记录的已知限制）	5
4	码本、影子参数与热更改	6
4.1	三层存储	6
4.2	影子参数 + 滞回翻转	6
4.3	热更改的原子性	6

5 共享基矩阵	7
5.1 参数量与低秩分解	7
5.2 在线成本	7
6 后端检索	7
6.1 只前 64 位参与检索	7
6.2 为何一期不用 MIH	7
7 训练路线	8
7.1 阶段 0: 语义锚点（第一个里程碑，不可跳过）	8
7.2 阶段 1: 码本 + 基矩阵联合训练	8
7.3 阶段 2: 位级编辑 + 遗忘 + 自学习闭环	8
8 未决项 D7: 激活二值化下的精度补偿	9
8.1 问题的形式	9
8.2 为何这对本架构特别关键	9
8.3 由此产生的三个子问题（需要作者决断）	9
9 Rust 引擎设计	9
9.1 模块划分	9
9.2 跨字 popcount 的实现要点	10
10 环境与资产	10
10.1 已实测的环境（全部本机验证）	10
10.2 网络（关键约束）	10
10.3 本地资产	10
10.4 已知坑	11
11 对 HSH-64 论文数学命题的核查	11
11.1 核查结果	11
11.2 Prop.3 的精确错误定位	11
11.3 影响评估	12
12 风险清单	12
13 里程碑与验收标准	13
14 决策点汇总	13
15 待确认清单	14

1 定位与总览

1.1 一句话定位

把 HSH-64 的 64 位结构化语义码, 从「只读检索键」扩展为「可读写、可被反向传播直接编辑的权重参数载体」: 在原 64 位检索码后追加一段不参与检索的 per-token 参数段, 使每个 token 获得唯一的权重矩阵; 权重由「共享基矩阵」按码指定的符号模式组合而成。

这套设计的三个卖点不是并列的, 而是同一个位运算的三种表现:

能力	在标准 Transformer 里
构建权重	全局权重矩阵, 与 token 无关
语义检索	独立的外部向量库 + 独立编码器
参数热更改	微调 / 模型编辑, 需重算浮点权重

而在本设计中, 这三件事退化为同一个位操作: 翻转 1 bit 即同时改变该 token 的权重、改变它的检索近邻、并完成一次可原子提交的更新。

改变它的检索近邻、并完成一次可原子提交的更新。

1.2 与 v1 的差异

v1 版本假设「主干为冻结的全精度模型, 只在其上挂一个二值码本」。该假设已作废——作者的设定是: 激活走二值化, 权重允许浮点, 且不存在全局权重, 参数直接对应到每个 token。本版据此重写。

1.3 HSH-64 原始设计（作者论文）

本设计完全建立在作者的 HSH-64 之上, 其原始约定为:

字段	位宽	原语义（检索语境）
feat	4	词性 / 语义类别, 最多 16 类, 可作硬过滤
sim	52	语义相似码, 主导检索质量; 容量 $2^{52} \approx 4.5 \times 10^{15}$
abs	8	簇内完美哈希标识, 用于区分同 (feat, sim) 桶内的词
合计	64	单个 u64 存储, 一次 popcnt 计算 Hamming 距离

关键性质(v1 曾误解, 本版修正): abs 是桶内唯一编号, 不是桶的地址。桶的地址由 (feat, sim) 决定。这一区分决定了容量结论（见 §3.3）。

2 参考实现：快照级约定

前置声明: 以下全部结论来自 F:\HSH_64 的快照源码阅读, 并通过 `cargo test --release` (21/21 通过, 0.03s) 验证其自洽性。作者已确认该快照滞后于最新代码。拿到权威版本后, 下表各项必须复核。

需复核项	当前快照结论	风险
位布局	feat[63:60] + sim[59:8] + abs[7:0], SIM_SHIFT=8	高
模型文件格式	魔数 0xCAB1DE3D, 全 big-endian , v1 头 20B / v2 头 24B	高
Deep Hash 结构	$\mathbf{x} \rightarrow [\text{Linear} \rightarrow \text{BN}(\text{eval}, \text{eps}=1\text{e-}5) \rightarrow \text{ReLU}] \times \text{depth} \rightarrow \text{Linear}$	中
量化判定	sim 第 i 位置 $1 \iff u_i \geq 0$ (含零)	中
MIH 自适应停止	$ C \geq \max(K, \alpha K)$ 且 $ C_{\text{new}} / C < \delta$	中
贪心翻转	$\Delta_{i,b} = \sum_{j \neq i} W_{ij}(1 - 2 \cdot \mathbb{K}[C_{i,b} = C_{j,b}])$	中

2.1 Deep Hash 二进制格式（导出侧必须逐字节对齐）

偏移	字段	类型
0	magic = 0xCAB1DE3D	u32 BE
4	version (1 或 2)	u32 BE
8	dim	u32 BE
12	hidden_dim	u32 BE
16	depth (仅 v2)	u32 BE
16 或 20	n_bits	u32 BE
...	mean	dim × f32 BE
...	每层 $W, b, \gamma, \beta, \mu, \sigma^2$	f32 BE
...	$W_{\text{out}}, b_{\text{out}}$	f32 BE

3 权重码架构（本设计的核心）

3.1 128 位权重码

在 HSH-64 的 64 位检索码之后，追加 64 位不参与检索的 per-token 参数段：

feat(4)	sim(52)	abs(8)	param(64)
hash 段（64 位）：参与检索		参数段（64 位）：仅用于权重生成	

param 段存在的必然性：§3.3 将说明 (feat, sim) 定义语义桶、abs 是桶内唯一编号。因此同一个桶内的 token 共享 (feat, sim) 与全部 sim 位，在原 64 位内不存在任何可用于区分它们权重的自由度。必须有新的一段。

3.2 三层语义对齐（「无缝衔接」的精确定义）

作者要求「后端到前端的哈希码含义完全无缝衔接」。在本架构下，三个层次是同一个划分：

层次	后端（检索）	前端（权重）	是否同一划分
(feat,sim)	语义桶地址	选择共享基矩阵 B_j	同一个
abs	桶内唯一编号	桶内 token 消歧	同一个
param	不参与	per-token 符号调制	新增自由度
整个 128 位	（仅前 64 位）	权重标识	见 §6

无转换层：检索用的桶就是权重用的基，检索用的 abs 就是 token 消歧编号。

3.3 容量分析（修正 v1 的错误结论）

v1 的错误： v1 曾把 abs 当作桶地址，得到「桶数 = $2^{F+A} = 4096 < 151,936$ 」，进而推出「需要动态码长 / 变长码 / 分层检索」等一整套复杂机制。该前提错误，相关机制全部不需要。

正确算法如下。

命题 1 (桶结构与容量). 设 hash 段为 $feat(F) + sim(S) + abs(A)$ ，词表大小 N 。则

1. 语义桶 (semantic bucket) 数为 2^{F+S} ，不是 2^{F+A} ；
2. 每桶可用 abs 槽位为 $2^A = 256$ ；
3. 全局容量为 $2^{F+S} \cdot 2^A = 2^{64}$ 。

代入 $F = 4, S = 52, A = 8$:

语义桶数	$2^4 \times 2^{52} = 2^{56} \approx 7.2 \times 10^{16}$
每桶槽位	$2^8 = 256$
全局容量	$2^{64} \approx 1.8 \times 10^{19}$
实际需求 (Qwen2.5 词表)	151,936
冗余	$\approx 1.2 \times 10^{14}$ 倍
平均占用	$151,936 / 2^{56} \approx 37 \text{ token/桶} \ll 256 \checkmark$

注 1 (唯一性判据). abs 提供完美哈希的充分必要条件是：每个 (feat,sim) 桶内的 token 数 ≤ 256 。桶数不需要为 $N/256$ ，因为 (feat,sim) 的空间本身极大。

唯一残留风险： 桶大小的分布。若某桶堆入 > 256 个 token，abs 溢出。该量可直接测量：对 Qwen 词表跑一遍投影，统计 (feat,sim) 桶大小直方图。若最大桶 < 256 ，则本架构原样成立，一位都不需修改。

3.4 权重生成

定义 1 (权重生成). 第 ℓ 层、token t 的权重矩阵为

$$W_t^{(\ell)} = \sum_{j=1}^k s_j(\text{sim}_t, p_t) \cdot B_j^{(\ell)} \quad (1)$$

其中 $B_j^{(\ell)} \in \mathbb{R}^{d \times d}$ 是共享基矩阵， $s_j \in \{-1, +1\}$ 为符号系数。

3.4.1 符号合成: $s = \text{mask} \odot p$

定义 2 (符号合成).

$$\text{mask}(\text{sim}_t) \in \{-1, +1\}^{52}, \quad \text{mask}_j = \begin{cases} +1, & \text{sim 第 } j \text{ 位} = 1 \\ -1, & \text{sim 第 } j \text{ 位} = 0 \end{cases} \quad (2)$$

$$p_t \in \{-1, +1\}^{64}, \quad p_t \text{ 由 param 段给出} \quad (3)$$

$$s_j = \text{mask}_j(\text{sim}_t) \cdot p_t[j \bmod 64] \quad (4)$$

3.4.2 几何隔离性质 (本设计的数学基石)

定理 1 (几何隔离). 对两个 token a, b , 其符号模式之差的 *Hamming* 重量满足

$$|\{j : s_j(a) \neq s_j(b)\}| = |\{j : \text{mask}_j(a) \neq \text{mask}_j(b)\}| \oplus |\{j : p_a[j] \neq p_b[j]\}| \quad (5)$$

特别地, 当二者属于同一语义桶 ($\text{sim}_a = \text{sim}_b$) 时:

$$|s(a) \odot s(b)| = d_H(p_a, p_b) \quad (6)$$

即权重差异完全由 *param* 决定, 与共享的 *mask* 无关。

证明. $s(a) \odot s(b) = (\text{mask}(a) \odot \text{mask}(b)) \odot (p_a \odot p_b)$ 。当 $\text{mask}(a) = \text{mask}(b)$ 时第一项为全 +1, 故只剩 $p_a \odot p_b$ 。一般情形下逐位异或, 重量为两部分的逐位异或, 即所述。□

注 2 (工程含义). 改 *param* 不会破坏 *sim* 建立的语义结构, 反之亦然。这是可证明的隔离, 而非依靠超参保证。对「热更改」而言这是安全性保证: 可以只动 per-token 参数而不扰动语义几何, 也可以只动语义码而不影响 token 身份。

3.5 param 的结构化初始化

若 p_t 为白噪声, 则同桶 token 的权重互不相关, 语义聚簇失效。因此需要结构化初始化:

初始化	后果
逐位随机	同桶 token 差异过大 $\Rightarrow$ 语义聚簇失效
全常数	同桶 token 几乎相同 $\Rightarrow$ 256 个槽位浪费
块常数 (建议块宽 8)	块内共享 $\Rightarrow$ 保留局部结构; 块间不同 $\Rightarrow$ 提供区分度

块宽 (默认 8, 即 8 个块) 是本设计唯一需要调的结构超参。

3.6 表达力边界 (须记录的已知限制)

由式(1), $s \in \{-1, +1\}^k$ 只能取 k 个基的符号组合。因此每个 token 的权重被限制在「基矩阵的符号翻转」这一集合内: 幅值完全由基决定, token 无法调整自身权重的大小, 只能调方向。

这在数量上充足 (每桶 256 个 token, 可用权重矩阵数 $\leq 2^{64}$), 但若实验表明需要 per-token 幅值自由度, 可在 *param* 之外追加一小段浮点 scale (4 ~ 8 个 fp16)。该小段不参与检索, 故不破坏定理 1 的隔离性。建议先不加, 由实验决定。

4 码本、影子参数与热更改

4.1 三层存储

结构	形状	精度	大小 ($N=152k$)	更新频率
CodeTable	$N \times 128$ bit	$2 \times u64$	2.43 MB	慢（滞回翻转）
Shadow	$N \times 128$	fp16	39 MB	快（每步梯度）
Basis	k 个 $d \times d$	fp16/低秩	见 §5	每步梯度

4.2 影子参数 + 滞回翻转

为什么必须有影子参数：若直接对码位做 STE，会出现两个致命问题——

1. 低频 token 学不到：batch 只覆盖约 3 万 token，词表 15 万，低频 token 永不进入 batch $\Rightarrow$ 永不更新。
2. 错位无法纠正：STE 梯度只沿当前码的 ± 1 方向，码已错则梯度把它推得更错——无法翻转一位。

定义 3 (影子参数与滞回翻转). 对每个 token 的每一位维护连续影子参数 $\theta_{t,b} \in \mathbb{R}$ ，码位为 $c_{t,b} = \text{sign}(\theta_{t,b})$ ，前向使用 c ，反向 STE 令 $\partial c / \partial \theta \approx \mathbb{I}$ 。仅当

$$|\theta_{t,b}| > \tau_{\text{flip}} \quad \text{且} \quad \text{sign}(\theta_{t,b}) \neq c_{t,b} \quad (7)$$

时才实际翻转该位。 τ_{flip} 由 1.0 线性退火至 0.3。

额外收益： $|\theta_{t,b}|$ 就是该位的置信度，可直接用于 HSH-64 的非对称距离评分，无需额外计算。

4.3 热更改的原子性

```
// 读者路径：无锁、无等待
let snap: Arc<Snapshot> = self.snapshot.load(Ordering::Acquire);
let codes: &[u64] = &snap.codes; // 不可变切片，读者永不阻塞

// 写者路径：copy-on-write
let mut new_codes = snap.codes.clone(); // 只 clone 受影响的桶
new_codes[2*t] ^= mask_lo; // 128 位 = 2*u64
new_codes[2*t + 1] ^= mask_hi;
let mut new_snap = Snapshot::from(snap);
new_snap.codes = Arc::new(new_codes);
new_snap.version = snap.version + 1;
self.snapshot.store(Arc::new(new_snap)); // 一次原子指针交换
```

保证：读者不阻塞、不加锁；单次翻转对所有并发读者原子可见（无中间态）；回滚 = 存回旧 Arc, $O(1)$ 。

并发安全边界： $\text{new_codes}[..] \hat{=} \text{mask}$ 操作的是 clone 出来的私有副本，因此不需要 CAS 循环。直接原地修改 $\&[u64]$ 是 UB。此项须在代码审查中重点确认。

5 共享基矩阵

5.1 参数量与低秩分解

方案	每基参数	k 基合计	可行性
完整 $d \times d$ fp16	802k	51 M ($k=64$)	可接受
低秩 $r=32$	57k	3.7 M ($k=64$)	推荐

建议 $B_j = U_j V_j^\top$, $U_j \in \mathbb{R}^{d \times r}$, $V_j \in \mathbb{R}^{r \times d}$, $d = 896$ 、 $r = 32$ 、 $k = 64$ 时基参数约 **370 万**, 在 8 GB 显存下宽裕。

5.2 在线成本

每个 token 需 k 次秩 r 的应用 (式(1) 的符号已由 popcount 类运算得到):

$k = 64, r = 32, d = 896$	$64 \times (2 \times 896 \times 32) = 3.67 \times 10^6$ MACs / token
每 token 输出	$d = 896$ 维
与全连接 $d \times d$ 相比	3.67M vs 0.80M MACs (约 4.6 ×)

注意: 式(1) 的朴素实现比单个全连接层**更贵**。若在线成本成为瓶颈, 需要在以下方向优化 (见 §12 R6): 减少 k 、降低秩 r 、或仅对前若干层使用本机制。

6 后端检索

6.1 只前 64 位参与检索

方案	优点	缺点
全 128 位参与 仅前 64 位 (采纳)	一次大比较 语义几何纯净	param 污染语义距离: 语义相同的 token 会被参数差异拉远 跨字需两次 popcount

跨字成本:

$$d_H(a, b) = ((a_0 \oplus b_0).count_ones()) + ((a_1 \oplus b_1).count_ones()) \quad (8)$$

即两次 popcount 加一次加法, 仍是常数时间。在 15 万条规模下 ($< 20 \mu s$) 不构成瓶颈, 而语义几何纯净性的收益远大于省一次 popcount。

6.2 为何一期不用 MIH

HSH-64 的 MIH 分段设计 ($B = 13$, 每段 4 位) 是针对 $N = 3109$ 的。按论文定理 4 的期望候选数公式

$$\mathbb{E}[|C|] \leq N \left(1 - \left(1 - \frac{V(M/B, \lfloor r/B \rfloor)}{2^{M/B}} \right)^B \right) \quad (9)$$

在 $N = 150k$ 、每段 8 位时：半径 $r \leq 7$ 的候选池恒为约 1.1×10^4 （退化为 $O(N)$ ），而 $r = 8$ 时突然爆到约 8.4×10^4 （**超过暴力扫描**），且索引本身需约 62 MB，使内存膨胀 26 倍。

一期决策：使用暴力 **popcount**。15 万条 $\times$ 8 字节 = 1.2 MB 码本可完全驻留 L2/L3；SIMD 一次处理 4~8 个 **u64**，预期 $< 20 \mu s$ 。MIH 模块**保留**（可直接复用快照的 **mih_index.rs**），以 feature flag 切换，待 $N > 10^6$ 时启用。

7 训练路线

7.1 阶段 0：语义锚点（第一个里程碑，不可跳过）

1. 用 `F:\hsh64_models\bge-large-zh-v1.5` 对 `vocab_3109.txt` 生成语义嵌入。
2. 训练 Deep Hash 投影头，完全按 **HSH-64 论文 §5.2.2**： $\mathcal{L}_{\text{info}} + \lambda_q \mathcal{L}_{\text{quant}} + \lambda_b \mathcal{L}_{\text{balance}}$ ，STE，Adam， $\text{lr} = 10^{-3}$ 。
3. 用论文 Algorithm 1 做召回导向贪心后处理。
4. 评测纯 Hamming Recall@10。

验收： $\text{Recall@10} \geq 0.70$ （论文：h256 为 0.7382，h512 为 0.7404）。

为何不可跳过：若跳过此步，码是随机的，反传只能学到噪声，后续所有新架构实验无法归因。

7.2 阶段 1：码本 + 基矩阵联合训练

冻结语义锚点，训练 B_j 与 param 段：

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda_1 \mathcal{L}_{\text{quant}} + \lambda_2 \mathcal{L}_{\text{balance}} + \lambda_3 \mathcal{L}_{\text{iso}} \quad (10)$$

其中 $\mathcal{L}_{\text{iso}}$ 用于维持定理 1 的隔离性（惩罚 **mask** 与 p 的非正交耦合）。

7.3 阶段 2：位级编辑 + 遗忘 + 自学习闭环

离线编辑：复用论文 Algorithm 1，但目标函数从「检索召回」换成语言建模损失，此时的 $\Delta_{t,b}$ 没有闭式解，退化为有限差分：

$$\Delta_{t,b} = \mathcal{L}(\text{翻转第 } t \text{ 个 token 的第 } b \text{ 位}) - \mathcal{L} \quad (11)$$

成本为 batch 内唯一 token 数 U 乘 128 次前向，**必须批处理摊销**（把所有候选翻转堆成一个 batch 并行前向）。

遗忘系数（per-token，时间衰减）：

$$\theta_t \leftarrow \theta_t \cdot \lambda_t^{\Delta_t}, \quad \lambda_t = \lambda_{\text{base}}^{1+\nu \cdot \text{freq}[t]}^{-1} \quad (12)$$

当 $|\theta_{t,b}|$ 因衰减跌破 τ_{freeze} 时，该位**永久冻结**（长期记忆固化）。

自学习闭环（单跳知识问答）：

写入 陈述句 $\rightarrow$ 128 位权重码 $\rightarrow$ 后端存条目

查询 只给问题 $\rightarrow$ 投影出 hash 段 $\rightarrow$ popcount 检索 $\rightarrow$ 返回 value

指标 Hit@1 / Hit@10

验收：写入 500 条合成知识，Hit@10 ≥ 0.60 ，且基座 PPL 退化 $< 2\%$ 。

8 未决项 D7：激活二值化下的精度补偿

这是本文档唯一可能影响架构可行性的未决项。

8.1 问题的形式

已知设定：激活走二值化，权重允许浮点。前向为

$$b^{(l+1)} = \text{sign}(W_t^{(l)} b^{(l)}), \quad b^{(l)} \in \{-1, +1\}^d \quad (13)$$

8.2 为何这对本架构特别关键

本架构的全部精度来自浮点基矩阵 B_j 的线性组合。但二值激活下 $\text{sign}(W_t b)$ 只关心 $W_t b$ 的符号，浮点精度仅在与 0 比较的边界附近有意义。

具体地，“feat/abs 选哪组基”的作用退化为「移动决策边界的位置」；浮点基相对二值基的精度优势被激活的 sign 大幅抹平。

叠加低秩 ($r = 32$) 后问题更明显： $W_t b$ 落在 32 维子空间内，精度提升空间本就有限。

8.3 由此产生的三个子问题（需要作者决断）

ID	子问题	说明
D7a	二值激活的目的	若是「计算与检索统一」（激活 = 决策位，匹配 = popcount），则基也应走低比特，整个栈统一为位运算；若是「省推理算力」，则浮点权重与之矛盾
D7b	scale 机制	标准二进制网络靠 scaling factor 找回幅度。本架构中该 scale 是 per-token 的还是共享的？共享则不属于「全局权重」，per-token 则需额外存储
D7c	层间复合退化	若每层严格 sign ，两层线性变换在恒等区可合并，表达力可能退化。是否需要逐元素非线性或残差旁路？ 此项待实验验证，不作推测性结论

9 Rust 引擎设计

9.1 模块划分

```
crates/
  hsh-core/      # 复用快照 hsh64.rs / perfect_hash.rs / pos_map.rs (逐位兼容)
  hsh-memory/    # 码本 (2×u64)、暴力检索(SIMD)、写入、快照
  bnn-weights/   # 权重生成: mask p、基矩阵组合、低秩应用
  bnn-model/     # 前向: BitLinear、Attention、RMSNorm、KV Cache
  hsh-runtime/   # 热替换运行时: ArcSwap 双缓冲、版本号、回滚、mmap
  hsh-cli/       # bench / serve / codec 工具
```

9.2 跨字 popcount 的实现要点

```
#[inline(always)]
fn hamming128(a: (u64, u64), b: (u64, u64)) -> u32 {
    (a.0 ^ b.0).count_ones() + (a.1 ^ b.1).count_ones()
}
```

由于检索只用 hash 段，实际热路径为 `hamming64`，与快照实现完全一致。

10 环境与资产

10.1 已实测的环境（全部本机验证）

项	实测值	状态
Python	3.11.9	✓
PyTorch	2.5.1+cu121	✓
transformers / sentence-transformers	5.13.1 / 5.6.0	✓
Rust	cargo / rustc 1.96.0	✓
GPU	RTX 4060 Laptop, 8 GB, driver 616.64	<code>cuda.is_available()</code>
LaTeX	MiKTeX, xelatex + ctexart	✓ 中文与公式编译通过

10.2 网络（关键约束）

<code>huggingface.co</code>	超时不可达
<code>hf-mirror.com</code>	可用（已实测可下载）
<code>modelscope.cn</code> / <code>pypi.org</code>	可用

所有模型下载必须走 `HF_ENDPOINT=https://hf-mirror.com`。

10.3 本地资产

路径	内容	状态
<code>F:\HSH_64</code>	论文参考实现（Rust 8 模块 + 12 Python 脚本）	旧快照
<code>...\tests\data\vocab_3109.txt</code>	3109 词表	✓
<code>F:\hsh64_models\bge-large-zh-v1.5</code>	教师权重 1.3 GB（392 张量）	✓
<code>E:\Ollama\Models</code>	qwen3:8b (Q4_K_M)	不可用作教师

Ollama 不可用作教师的实测依据：`/api/embed` 返回 501 Not Implemented, `/api/show` 仅返回元数据。GGUF 4-bit 量化权重无法提供 logits 或隐层状态，而蒸馏、码对齐、语义锚点三处均硬依赖它们。

10.4 已知坑

ID	现象	对策
K1	torch 2.5.1 加载 bge-large 报「requires torch ≥ 2.6」	torch.load(..., map_location="cpu") 用默认参数可绕过（已实测）；建议一次性转 safetensors
K2	C: 盘仅剩 18.4 GB	所有模型与缓存放 E:/F:
K3	快照非最新代码	见 §2 复核清单

11 对 HSH-64 论文数学命题的核查

为判断「容量是否够用」，本文档对论文第 6 节的命题做了纯 CPU 数值复现。复现脚本：
tools/verify_paper_math.py → results/paper_math_verification.txt。

11.1 核查结果

命题	内容	结果
Prop.1	Hamming 球容量 $V(52, 10) \approx 2.5 \times 10^{10}$	一致（实为 2.04×10^{10} ，四舍五入）
Prop.2	$\mathbb{E}[d_H] = M/2$, $\text{Var} = M/4$	一致（26 与 3.6056）
Prop.3	称 $d \leq 18$ 时 $N \cdot V(52, d - 1) < 2^{52}$	错误，实际最大 $d = 14$
Thm.1	比特翻转梯度闭式解	代数正确，本设计直接复用

11.2 Prop.3 的精确错误定位

论文原文：「对 $d = 18$, $V(52, 17) \approx 1.3 \times 10^{12}$, $N \cdot V(52, 17) \approx 3.9 \times 10^{15} < 2^{52} \approx 4.5 \times 10^{15}$, 故理论上存在这样的码本。」

实测：

$$\begin{aligned} V(52, 17) &= 3.9481 \times 10^{13} && \text{（论文称 } 1.3 \times 10^{12}\text{）} \\ V(64, 17) &= 2.0926 \times 10^{15} \\ 3109 \cdot V(52, 17) &= 1.2275 \times 10^{17} && \text{vs } 2^{52} = 4.5036 \times 10^{15} \text{ （超出 27 倍）} \end{aligned}$$

根因： 1.3×10^{12} 这组数字的量级对应 $M = 64$ 的 Hamming 球，而非 $M = 52$ 。即 $V(M, d - 1)$ 一步误用了 64 位空间。旁证：论文 §4 使用的 $2^{52} \approx 4.5 \times 10^{15}$ 自身自洽，唯独 Prop.3 的球体积跳到了 64 位。

逐项重算 $M = 52, N = 3109$:

d	$V(52, d - 1)$	$N \cdot V$	是否 $< 2^{52}$
13	2.8719×10^{11}	8.9288×10^{14}	成立
14	9.2221×10^{11}	2.8671×10^{15}	成立（上界）
15	2.6912×10^{12}	8.3669×10^{15}	不成立

正确结论： $d \leq 14$ ，而非 $d \leq 18$ 。

11.3 影响评估

1. 不影响论文任何实验结论。Prop.3是纯存在性理论论证，Recall 数字全部来自真实实验，订正它不需重跑训练。
2. 订正后 52 位的选择更有说服力。 $(M, d) = (52, 14)$ 给出 $d/M = 0.269$ ，紧贴 Hamming 界上界 (≈ 0.288) 与 Varshamov–Gilbert 界下界 (≈ 0.213) 之间。
3. 修正后的界强化了本设计的动机：52位码在 $N = 3109$ 时仅能支持最小距离 14；扩到 15.2 万 token 后碰撞压力上升，这正是「码需可写、可增量编辑」的硬性理由。

12 风险清单

ID	风险	对策	级别
R1	纯位级 STE 导致低频 token 学不到、错位不可纠	§4 影子参数 + 滞回翻转（必选）	致命
R2	桶大小分布不均，某桶 > 256 致 abs 溢出	直接测量桶大小直方图；必要时桶内换 sim 分桶	高
R3	二值激活抹平浮点基精度 (§8)	D7 决断	高
R4	有限差分位编辑成本 $O(U \times 128)$ 次前向	候选翻转堆成 batch 并行前向	高
R5	在线权重生成比全连接贵约 $4.6\times$	减少 k 、降低 r 、或仅前若干层启用	中
R6	一期 MIH 无效	已定量论证 (§6) $\Rightarrow$ 暴力 pop-count	中
R7	快照非最新代码，误改无法回滚	只读引用，代码复制到本仓库并注明出处	中
R8	8 GB 显存不足	Qwen2.5-0.5B (fp16 约 1 GB) + 预缓存教师输出	中
R9	几何隔离性在训练中被破坏	加 $\mathcal{L}_{\text{iso}}$ 正则 + 每步监控 $ s(a) \odot s(b) $	中

13 里程碑与验收标准

#	里程碑	验收标准（可量化）	备注
M0	环境就绪	三模型可加载且前向形状正确	—
M1	论文复现	纯 Hamming Recall@10 ≥ 0.70	不可跳过
M2	Rust 记忆后端	与快照 benchmark 逐查询一致；报告 QPS	—
M3	热替换运行时	loom/miri 零数据竞争；写延迟 p99 $< 10\mu s$	—
M3.5	桶分布测量	(feat,sim) 桶大小直方图，最大桶 < 256	新增，决定 R2
M3.6	核心消融	同参数量下「128 位码 + 共享基」vs「直接 per-token d 维 embedding」	新增，决定论文成立与否
M4	码本 + 基联合训练	未见 token 码与锚点码平均 Hamming < 16 bit；PPL 退化 $< 2\%$	—
M5	自学习闭环	写 500 条，Hit@10 ≥ 0.60 ；3 域后域 1 不退化	—
M6	遗忘机制验证	有/无遗忘系数消融；长期未用 token 的码位固化率	—

关于 M3.6（新增）：这是判断架构是否成立的唯一关键实验。因为「per-token 可训练参数」在数学上就是 embedding 层——Transformer 的 embedding 表一直是 per-token 参数、一直由梯度更新、且只更新 batch 内出现的行。必须证明「128 位压缩 + 共享基」不只是 embedding 的低配版。

- 若赢或打平 $\Rightarrow$ 架构成立：同时买到 112 $\times$ 压缩 + 检索键复用 + 位级热更改。
- 若输 $\Rightarrow$ 定位需改为「以检索为中心的轻量方案」，而非通用参数容器。

14 决策点汇总

ID	决策点	建议	状态
D1	param 段形态（bit 段 vs 浮点向量）	bit 段（维持位运算统一）	已定
D2	总码宽	128 位（2 $\times$ u64 对齐）	已定
D3	param 是否参与检索	不参与（仅前 64 位）	已定
D4	符号合成方式	$s = \text{mask}(\text{sim}) \odot p$	已定
D5	param 初始化	块常数，块宽 8	待验证
D6	基矩阵 k 与秩 r	$k = 64$, $r = 32$ （起点）	待调参
D7	激活二值化的目的与 scale 机制	见 §8，需作者决断	未决

15 待确认清单

1. **D7 (§8)**: 激活二值化的目的是「计算与检索统一」还是「省推理算力」? 这决定浮点基是否仍然必要, 以及 scale 机制如何设计。唯一可能影响架构可行性的未决项。
2. **最新仓库代码**: 作者将提供。拿到后须按 §2 逐项复核, 特别是二进制格式与位布局——若已变更, §2 与 Rust 解析器需同步修改。
3. **Prop.3 订正补丁**: 是否需要一个落在 docs/ 下的 LaTeX patch (不修改 F:\HSH_64 原目录)?
4. **快照的协作方式**: 只读参考, 还是直接在其上迭代? 后者需把 §12 的目录结构改为 workspace 形式。

本文档所有环境探测结论均来自本机实跑。未经验证的技术判断已明确标注为「建议」或「待验证」, 未作推测性结论。